# 0040. Combination Sum II

dium | Category: Backtracking | LeetCode: https://leetcode.com/problems/combi

# Problem

- Candidates may contain duplicates; each element may be used once. Return unique combinations summing to target.
- Example: [10,1,2,7,6,1,5], target 8 → [[1,1,6],[1,2,5],[1,7],[2,6]].

# Intuition

- Combination Sum with two changes: recurse with i + 1 (no reuse) and skip duplicates at the same level (as in Subsets II).

# Brute Force

- Enumerate all subsets and keep those summing to target, deduped: O(n  2^n).

# Optimal Approach

- Sort.
- Dfs(start, rem): rem == 0 → record.
- For i from start: skip if i > start and c[i] == c[i-1]; break if c[i] > rem; recurse with i + 1.

# Worked Example

```
sorted [1,1,2,5,6,7,10], target 8.

1 -> 1 -> 6          [1,1,6]
1 -> 2 -> 5          [1,2,5]
1 -> 7               [1,7]
(second 1 at level 0 skipped)
2 -> 6               [2,6]
```

# Java Solution

```java
public class CombinationSumII {
    public List<List<Integer>> combinationSum2(int[] candidates, int target) {
        int[] c = candidates.clone();
        Arrays.sort(c);
        List<List<Integer>> res = new ArrayList<>();
        dfs(c, 0, target, new ArrayList<>(), res);
        return res;
    }

    private void dfs(int[] c, int start, int rem, List<Integer> path, List<List<Integer>> res) {
        if (rem == 0) { res.add(new ArrayList<>(path)); return; }
        for (int i = start; i < c.length && c[i] <= rem; i++) {
            if (i > start && c[i] == c[i - 1]) continue;
            path.add(c[i]);
            dfs(c, i + 1, rem - c[i], path, res);
            path.remove(path.size() - 1);
        }
    }
}
```

# Complexity

- Time: O(n  2^n) worst case. Space: O(n).

# Pitfalls

- Both rules are needed: i + 1 prevents reuse, the skip prevents duplicate combinations.